

EkS-terraform-gitops-platform (version simplifiée sans ALB)

Cluster Elastic Kubernetes running in AWS supported by CI/CD git platform

[Sommaire](#)

EkS-terraform-gitops-platform (version simplifiée sans ALB)	1
Cluster Elastic Kubernetes running in AWS supported by CI/CD git platform	1
0. Architecture	3
0.1 structure repository	3
0.2 Initialisation projet	4
0.2.1 Créer le repo GitHub	4
0.2.2 Cloner localement	4
0.2.3. Initialiser structure	4
0.2.4. Premier commit	4
0.2.5. Vérifier dans github	4
1. Terraform (EKS simple)	5
1.1 main.tf	5
1.2 Commandes manuelles :	6
1.3 Lister les ressources AWS	6
1.4 Vérifications dans AWS	6
2. Connexion cluster	6
2.1 verification nodes	6
2.2 vérification pods systems	7
2.3 Vérifier Node Groups (AWS)	7
3. Bootstrap	7
3.1 Installer Argo CD	7
3.2 Accès UI	7
4 Observability setup	7
4.1 Installation Prometheus + Grafana	7
4.2 Vérification	8
4.3 Accès Grafana	8
5. Application	8
6. Helm chart	8

6.1 deployment.yaml.....	8
6.2 service.yaml (NodePort).....	9
6.3 chart.yaml.....	9
6.4 values.yaml.....	10
7. ArgoCD.....	10
▶ Bootstrap GitOps (1 seule fois)	11
8. Pipelines CI/CD.....	11
9. Test application.....	11
9.1 Récupérer node IP.....	11
9.2 Accès :.....	11
10. Autoscaling.....	11
10.1 Emplacement du HPA.....	11
10.2 Contenu du fichier hpa.yaml.....	11
10.3 Prérequis OBLIGATOIRES.....	12
10.3.1 metrics-server.....	12
10.3.2 Requests CPU dans le Deployment.....	12
10.3.3 Déploiement (GitOps réel).....	12
10.3.4 Vérification.....	13
10.3.5 Test réel.....	13
10.3.6 Observer le scaling.....	13
11. Debug.....	13
12. Tests & validations.....	14
12.1 Objectifs.....	14
12.2 Test d'accès à l'application.....	14
12.2.1 Récupérer IP node.....	14
12.2.2 Accéder à l'application.....	14
12.3 Verifications des ressources Kubernetes.....	14
12.4 Test autoscaling (HPA).....	14
12.4.1 Générer du trafic.....	14
12.4.2 Observer le scaling.....	15
12.4.3 Si ça ne marche pas.....	15
12.5 Test de résilience.....	15
12.6 Debug (simulation incident).....	15

0. Architecture

Stack :

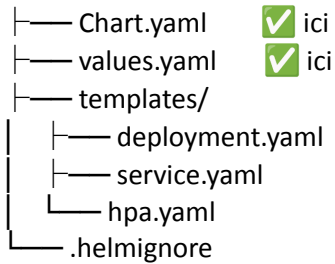
Amazon Web Services
Terraform
Amazon EKS
Argo CD
Helm
GitHub Actions

0.1 structure repository

eks-terraform-gitops-platform/

```
|
|├── terraform/
|│   ├── envs/prod/
|│   │   └── main.tf
|│
|├── app/
|│   └── Dockerfile
|
|├── helm-charts/myapp/
|│   ├── Chart.yaml
|│   ├── values.yaml
|│   └── templates/
|│       ├── deployment.yaml
|│       └── service.yaml
|
|├── argocd/app.yaml
|
|└── .github/workflows/
|   ├── terraform.yml
|   └── app.yml
```

```
helm-charts/
└── myapp/
```



0.2 Initialisation projet

0.2.1 Créer le repo GitHub

👉 sur GitHub :

nom : eks-terraform-gitops-platform

public ou privé

0.2.2 Cloner localement

git clone <https://github.com/YOUR-USERNAME/eks-terraform-gitops-platform>

cd eks-terraform-gitops-platform

0.2.3. Initialiser structure

Se placer dans le répertoire cloné par la commande git clone, précédente.

Cd eks-terraform-gitops-platform

echo "# GitOps Lab EKS" > README.md

Créer une structure de répertoires avec l'arborescence ci-dessus :

0.2.4. Premier commit

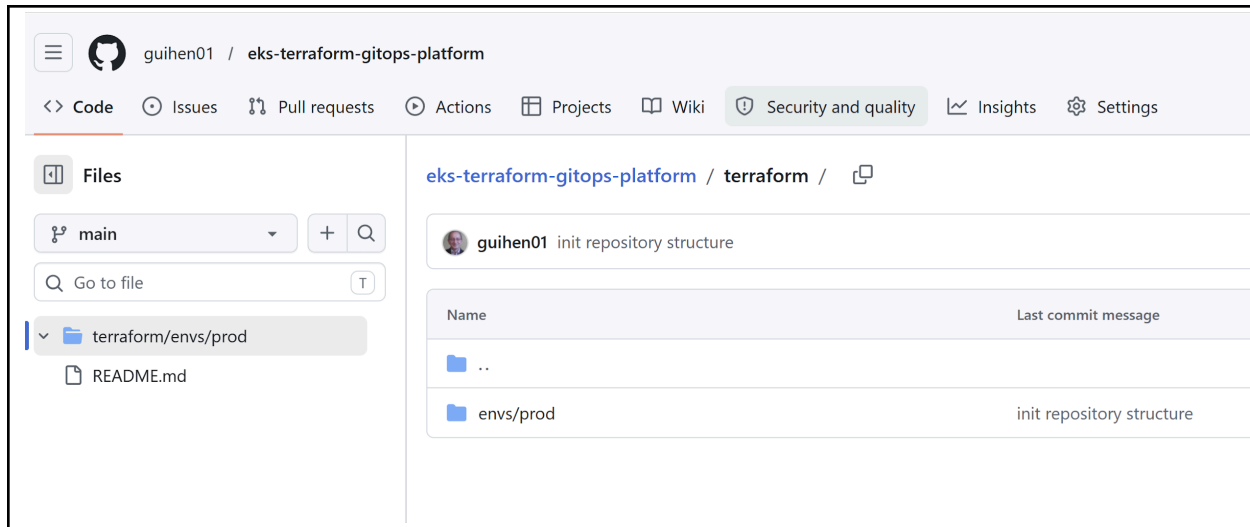
git add .

git commit -m "init repository structure"

git push

0.2.5. Vérifier dans github

<https://github.com/YOUR-USERNAME/eks-terraform-gitops-platform>



1. Terraform (EKS simple)

1.1 main.tf

 terraform/envs/prod/main.tf

```
provider "aws" {
  region = "ca-central-1"
}

module "eks" {
  source      = "terraform-aws-modules/eks/aws"
  version     = "~> 20.0"

  cluster_name   = "simple-platform"
  cluster_version = "1.28"

  vpc_id      = "vpc-0518bdeefb96a886d"
  subnet_ids = ["subnet-04ab7f22ddced88f9", "subnet-07b548eec9ff23574"]

  eks_managed_node_groups = {
    default = {
      desired_size = 2
      min_size     = 2
      max_size     = 3

      instance_types = ["t3.medium"]

      ami_type = "AL2_x86_64" # 🙌 AJOUT CRITIQUE
    }
  }
}
```

```

}

# 📌 AJOUT IMPORTANT (IAM → Kubernetes)
access_entries = {
  admin = {
    principal_arn = "arn:aws:iam::YOUR_ACCOUNT_ID:user/YOUR_USER"

    policy_associations = {
      admin = {
        policy_arn = "arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy"
        access_scope = {
          type = "cluster"
        }
      }
    }
  }
}
}
}
}
}

```

Récupérer sur votre compte AWS, un vpc_id, et au moins deux subnet_id

```

vpc-0518bdeefb96a886d
subnet-04ab7f22ddced88f9
subnet-07b548eec9ff23574

```

Récupérer l'ARN du compte AWS utilisé avec la commande :

```
aws sts get-caller-identity
```

puis coller l'ARN dans le code main.tf au niveau de l'instruction :

```
principal_arn = "arn:aws:iam::YOUR_ACCOUNT_ID:user/YOUR_USER"
```

1.2 Commandes manuelles :

```
cd terraform/envs/prod
```

```
terraform init
```

```
terraform plan
```

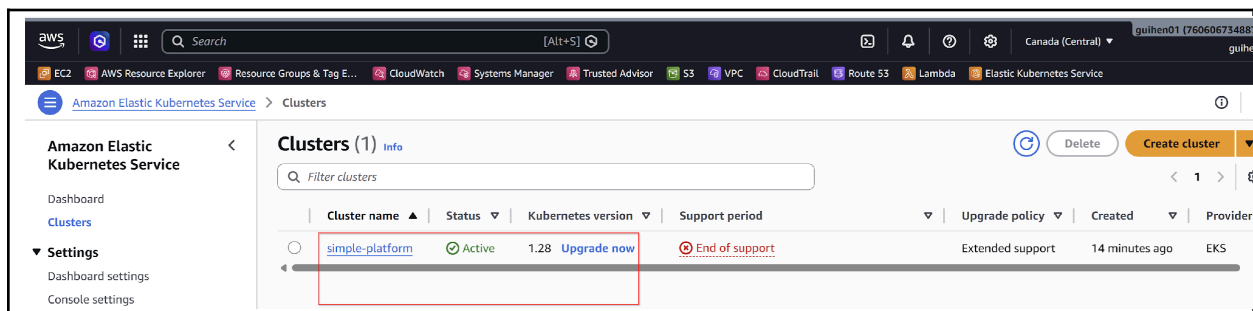
```
# module.eks.module.eks_managed_node_group["default"].module.user_data.null_resource.validate_cluster_service_cidr will be created
+ resource "null_resource" "validate_cluster_service_cidr" {
+   id = (known after apply)
+ }

Plan: 33 to add, 0 to change, 0 to destroy.

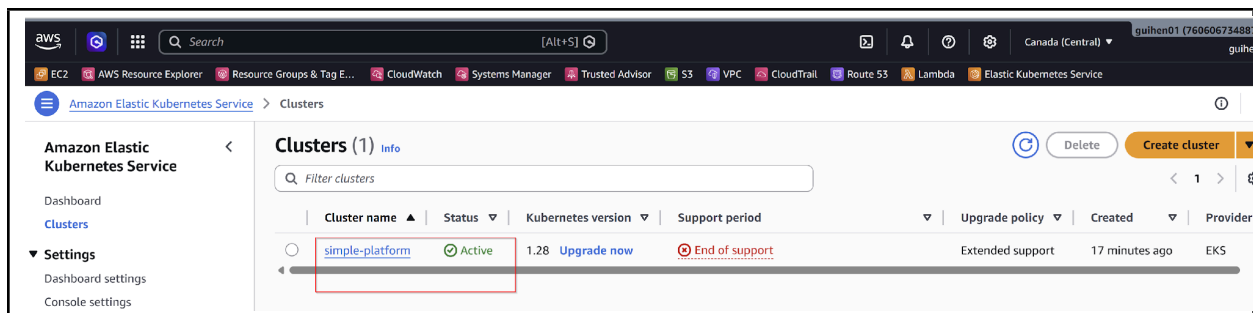
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
henri@Henri:~/labs/eks-terraform-gitops-platform/terraform/envs/prod$
```

terraform apply

lors de la phase de l'apply on peut vérifier dans la console AWS au niveau du menu de EKS que le cluster a été crée et qu'il est en cours de configuration :



L'installation est un peu longue. Ici , elle aura durée environ 15 minutes
On voit dans la figure ci-dessous que le cluster est actif dans AWS.



```
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Still creating... [01m18s elapsed]
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Still creating... [01m26s elapsed]
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Still creating... [01m36s elapsed]
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Still creating... [01m46s elapsed]
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Still creating... [01m55s elapsed]
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Still creating... [02m05s elapsed]
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Still creating... [02m15s elapsed]
module.eks.module.eks_managed_node_group["default"].aws_eks_node_group.this[0]: Creation complete after 2m22s [id=simple-platform:default-20260504212048497700000001]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
henri@Henri:~/labs/eks-terraform-gitops-platform/terraform/envs/prod$
```

1.3 Lister les ressources AWS

Lister les ressources AWS créées par terraform

terraform state list

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
henri@Henri:~/labs/eks-terraform-gitops-platform/terraform/envs/prod$ terraform state list
module.eks.data.aws_caller_identity.current[0]
module.eks.data.aws_iam_policy_document.assume_role_policy[0]
module.eks.data.aws_iam_policy_document.custom[0]
module.eks.data.aws_iam_session_context.current[0]
module.eks.data.aws_partition.current[0]
module.eks.data.tls_certificate.this[0]
module.eks.aws_cloudwatch_log_group.this[0]
module.eks.aws_eks_cluster.this[0]
module.eks.aws_iam_openid_connect_provider.oidc_provider[0]
module.eks.aws_iam_policy.cluster_encryption[0]
module.eks.aws_iam_policy.custom[0]
module.eks.aws_iam_role.this[0]
```

1.4 Vérifications dans AWS

Aller dans la console AWS et vérifier que les ressources décrites dans le fichier main.tf ont été créées

2. Connexion cluster

```
aws eks update-kubeconfig --region ca-central-1 --name simple-platform
```

2.1 verification nodes

kubectl get nodes

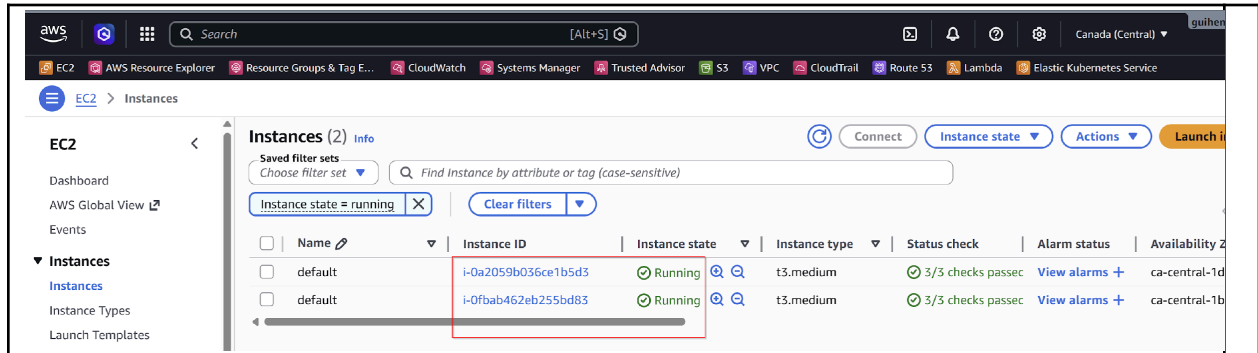
```
henri@Henri:~/labs/eks-terraform-gitops-platform/terraform/envs/prod$
henri@Henri:~/labs/eks-terraform-gitops-platform/terraform/envs/prod$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-172-31-15-216.ca-central-1.compute.internal Ready    <none>   49m   v1.28.15-eks-c39b1d0
ip-172-31-47-181.ca-central-1.compute.internal Ready    <none>   49m   v1.28.15-eks-c39b1d0
henri@Henri:~/labs/eks-terraform-gitops-platform/terraform/envs/prod$
```

remarque : kubectl is a local client that interacts with the EKS API server over the network

On lance la commande (kubectl get nodes) **depuis notre poste local**, kubectl contacte l'API server de Amazon EKS, AWS authentifie via IAM, le cluster renvoie la liste des nodes.

Les 2nodes tournent dans le cloud AWS

On peut voir que les 2 nodes correspondent aux deux instances EC2 qui tournent dans AWS .
Aller dans la console AWS , et on voit la figure ci-dessous :



2.2 vérification pods systems

kubectl get pods -n kube-system

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
aws-node-4t92b                     2/2     Running   0           51m
aws-node-zrb4s                     2/2     Running   0           51m
coredns-69458cc57b-kncz7           1/1     Running   0           59m
coredns-69458cc57b-ttrkt           1/1     Running   0           59m
kube-proxy-pxp7j                   1/1     Running   0           51m
kube-proxy-rl2xb                   1/1     Running   0           51m
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

👉 attendu : coredns - kube-proxy - aws-node

2.3 Vérifier Node Groups (AWS)

aws eks list-nodegroups --cluster-name simple-platform

```
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$ aws eks list-nodegroups --cluster-name simple-platform
{
  "nodegroups": [
    "default-20260504212048497700000001"
  ]
}
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

aws eks describe-nodegroup --cluster-name simple-platform --nodegroup-name <nodegroup-name>

👉 Vérifie : status: ACTIVE taille du groupe

3. Bootstrap

3.1 Installer Argo CD

kubectl create namespace argocd

helm repo add argo https://argoproj.github.io/argo-helm

helm install argocd argo/argo-cd -n argocd

Vérifier que argocd fonctionne :

kubectl get pods -n argocd

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           74s
argocd-applicationset-controller-7859547b7-sqmcp  1/1     Running   0           75s
argocd-dex-server-7cc5d657b9-hh9t2    1/1     Running   0           75s
argocd-notifications-controller-6ff476c9c5-9jzp9  1/1     Running   0           75s
argocd-redis-b478978db-p7s87         1/1     Running   0           75s
argocd-repo-server-54f6cccf54-kmrwv    1/1     Running   0           75s
argocd-server-5bffffd7b5d-xv2m6       1/1     Running   0           75s
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

3.2 Accès UI

kubectl port-forward svc/argocd-server -n argocd 8080:443

puis ouvrir : <https://localhost:8080>

mot de passe : admin

kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d && echo

YBYqvOaQd-N8a49u

```
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d && echo
YBYqvOaQd-N8a49u
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

4 Observability setup

4.1 Installation Prometheus + Grafana

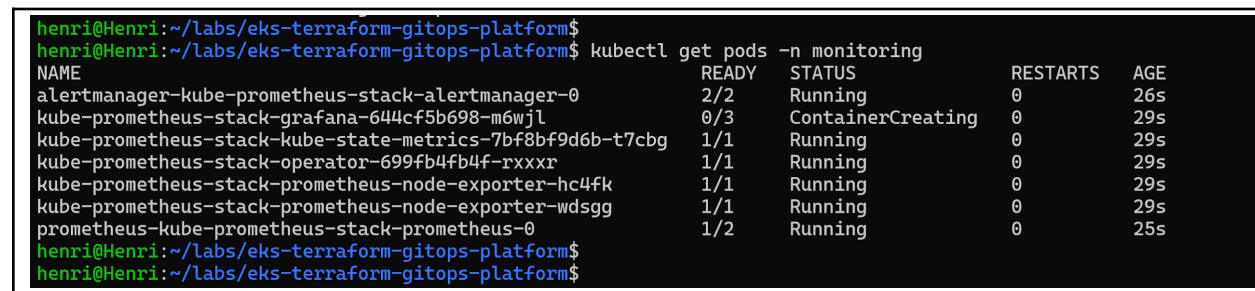
```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

```
kubectl create namespace monitoring
```

```
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack -n monitoring
```

4.2 Vérification

```
kubectl get pods -n monitoring
```



```
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-kube-prometheus-stack-alertmanager-0	2/2	Running	0	26s
kube-prometheus-stack-grafana-644cf5b698-m6wjl	0/3	ContainerCreating	0	29s
kube-prometheus-stack-kube-state-metrics-7bf8bf9d6b-t7cbg	1/1	Running	0	29s
kube-prometheus-stack-operator-699fb4fb4f-rxxxr	1/1	Running	0	29s
kube-prometheus-stack-prometheus-node-exporter-hc4fk	1/1	Running	0	29s
kube-prometheus-stack-prometheus-node-exporter-wdsgg	1/1	Running	0	29s
prometheus-kube-prometheus-stack-prometheus-0	1/2	Running	0	25s

```
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

4.3 Vérification générale de tous les pods

Afin de voir tous les pods présents , ce sur tous les namespaces
On reconnaitra les deux nodes AWS , ceux de argocd et ceux de prometheus

Kubectl get pods -A

```
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods -A
NAMESPACE      NAME                                                    READY   STATUS    RESTARTS   AGE
argocd          argocd-application-controller-0                       1/1     Running   0           38m
argocd          argocd-applicationset-controller-7859547b7-sqmcq      1/1     Running   0           38m
argocd          argocd-dex-server-7cc5d657b9-hh9t2                  1/1     Running   0           38m
argocd          argocd-notifications-controller-6ff476c9c5-9jzp9      1/1     Running   0           38m
argocd          argocd-redis-b478978db-p7s87                         1/1     Running   0           38m
argocd          argocd-repo-server-54f6cccf54-kmrwv                  1/1     Running   0           38m
argocd          argocd-server-5bffffd7b5d-xv2m6                     1/1     Running   0           38m
kube-system     aws-node-4t92b                                         2/2     Running   0           99m
kube-system     aws-node-zrb4s                                         2/2     Running   0           100m
kube-system     coredns-69458cc57b-kncz7                             1/1     Running   0           108m
kube-system     coredns-69458cc57b-ttrkt                             1/1     Running   0           108m
kube-system     kube-proxy-pxp7j                                       1/1     Running   0           100m
kube-system     kube-proxy-rl2xb                                       1/1     Running   0           99m
monitoring      alertmanager-kube-prometheus-stack-alertmanager-0    2/2     Running   0           2m7s
monitoring      kube-prometheus-stack-grafana-644cf5b698-m6wjl       3/3     Running   0           2m10s
monitoring      kube-prometheus-stack-kube-state-metrics-7bf8bf9d6b-t7cbg 1/1     Running   0           2m10s
monitoring      kube-prometheus-stack-operator-699fb4fb4f-rxxxr      1/1     Running   0           2m10s
monitoring      kube-prometheus-stack-prometheus-node-exporter-hc4fk  1/1     Running   0           2m10s
monitoring      kube-prometheus-stack-prometheus-node-exporter-wdsgg  1/1     Running   0           2m10s
monitoring      prometheus-kube-prometheus-stack-prometheus-0       2/2     Running   0           2m6s
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

4.4 Accès grafana (username/password)

kubectl get secret -n monitoring kube-prometheus-stack-grafana \

-o jsonpath="{.data.admin-password}" | base64 -d && echo

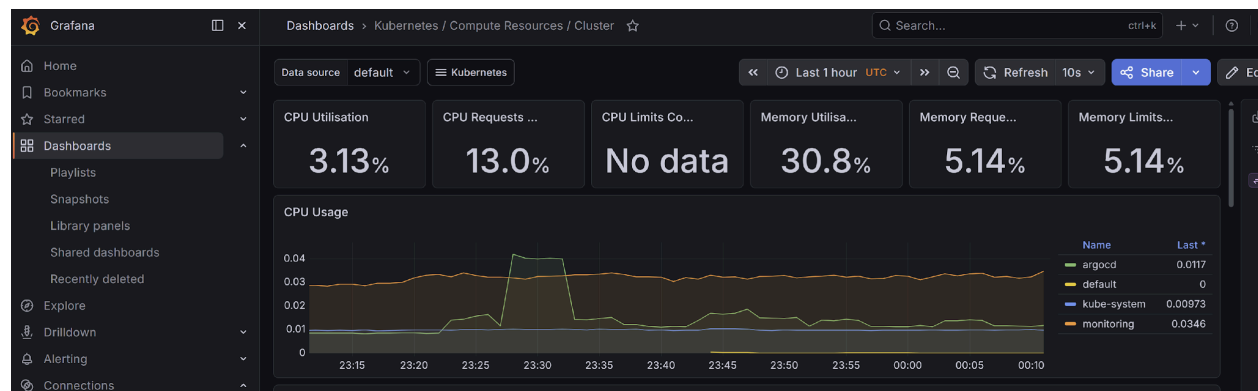
```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get secret -n monitoring kube-prometheus-stack-grafana \
-o jsonpath="{.data.admin-password}" | base64 -d && echo
e8zNDhFAHw4UqjHZtqxRezY4ROTj2xfj2zHpkF1v
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

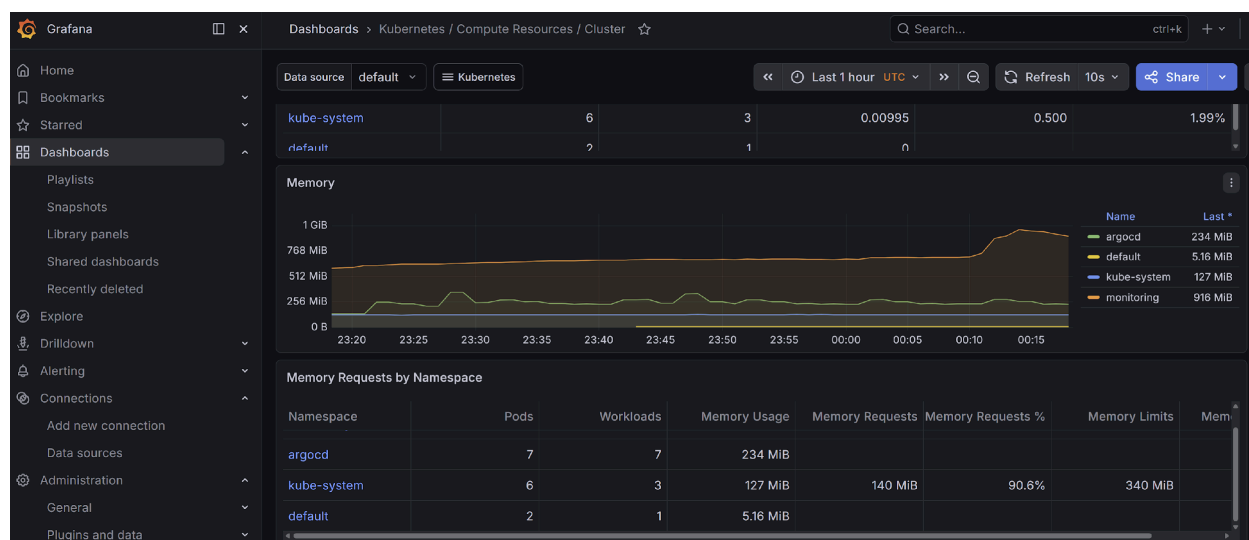
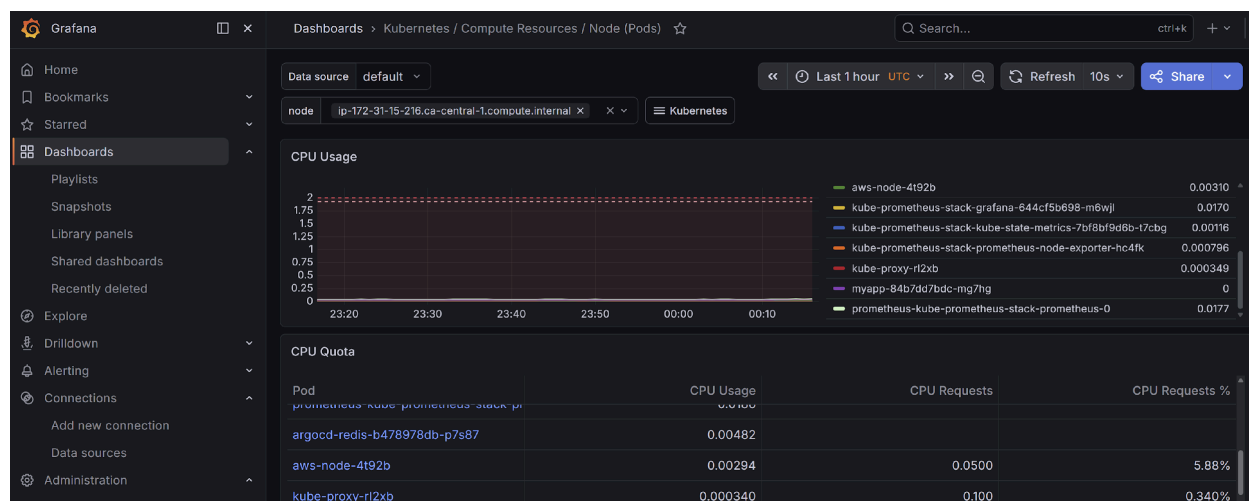
e8zNDhFAHw4UqjHZtqxRezY4ROTj2xfj2zHpkF1v

kubectl port-forward svc/kube-prometheus-stack-grafana -n monitoring 3000:80

Puis : <http://localhost:3000>

Image du cpu sur le cluster :





5. Application

app/Dockerfile

FROM nginx

6. Helm chart

6.1 deployment.yaml

helm-charts/myapp/templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
          ports:
            - containerPort: 80
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

```
git add .
git commit -m "add helm chart for myapp"
git push
```

6.2 service.yaml (NodePort)

helm-charts/myapp/templates/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  type: NodePort
  selector:
    app: myapp
```

```
ports:
  - port: 80
    targetPort: 80
    nodePort: 30007
```

```
git add .
git commit -m "add service to helm chart"
git push
```

6.3 chart.yaml

Créer le fichier dans le répertoire : helm-charts/myapp/

```
apiVersion: v2
name: my-app
description: Simple app deployed via Helm and ArgoCD
type: application
version: 0.1.0
appVersion: "1.0"
```

6.4 values.yaml

Créer le fichier values.yaml dans le répertoire : helm-charts/myapp/

```
replica Count: 2

image:
  repository: nginx
  tag: latest

service:
  type: ClusterIP
  port: 80

resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "200m"
    memory: "256Mi"
```

7. ArgoCD

 argocd/app.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
spec:
  source:
    repoURL: https://github.com/your-repo/eks-terraform-gitops-platform
    path: helm-charts/myapp
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated: {}
```

```
git add .
git commit -m "add argocd application"
git push
```

Bootstrap GitOps (1 seule fois)

```
kubectl apply -f argocd/app.yaml
```

puis :

```
kubectl get applications -n argocd
```

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get applications -n argocd
NAME      SYNC STATUS   HEALTH STATUS
myapp     Synced      Healthy
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

Kubectl get nodes

```
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get nodes
NAME                                                    STATUS    ROLES    AGE   VERSION
ip-172-31-15-216.ca-central-1.compute.internal        Ready     <none>   134m  v1.28.15-eks-c39b1d0
ip-172-31-47-181.ca-central-1.compute.internal        Ready     <none>   134m  v1.28.15-eks-c39b1d0
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

Kubectl get pods


```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myapp-84b7dd7bdc-7t2qc             1/1     Running   0           3m36s
myapp-84b7dd7bdc-mg7hg             1/1     Running   0           3m36s
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

On voit les 2 pods myapp_.... de notre application qui ont été déployés par argocd

kubectl get svc

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get svc
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
kubernetes    ClusterIP   10.100.0.1     <none>       443/TCP    156m
myapp         ClusterIP   10.100.184.192 <none>       80/TCP     6m1s
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

On voit le service myapp qui est présent

8. Pipelines CI/CD

Possibilité d'automatiser l'infrastructure avec terraform, git et github actions

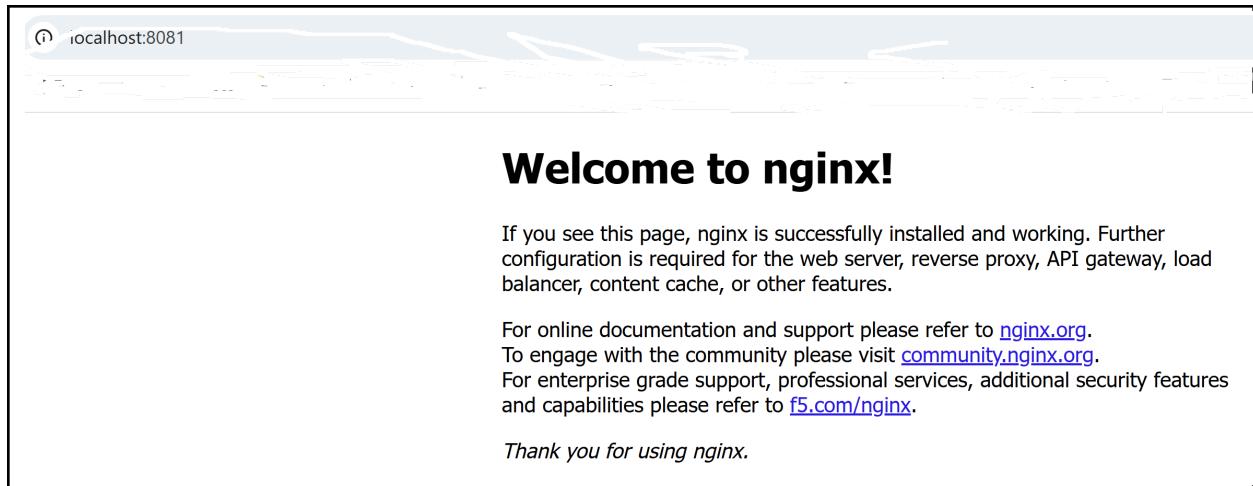
Mais on ne le fera pas dans ce lab, on fera de l'automatisation dans des futurs labs, avec
.github/workflows/terraform.yml

9. Test application myapp

kubectl port-forward svc/myapp 8081:80

puis : <http://localhost:8081>

on doit avoir : Welcome to nginx!



10. Autoscaling

10.1 Emplacement du HPA

helm-charts/myapp/templates/hpa.yaml

👉 même niveau que :
deployment.yaml
service.yaml

👉 logique :
tout ce qui est Kubernetes applicatif = Helm + GitOps

10.2 Contenu du fichier hpa.yaml

Hpa.yaml :

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
```

```
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 50
```

10.3 Prérequis OBLIGATOIRES

10.3.1 metrics-server

kubectI apply -f
<https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml>

👉 sans ça :
❌ HPA ne fonctionne pas

10.3.2 Requests CPU dans le Deployment

Rajouter dans la configuration du fichier deployment.yaml : le code suivant :

```
resources:
  requests:
    cpu: {{ .Values.resources.requests.cpu }}
    memory: {{ .Values.resources.requests.memory }}
  limits:
    cpu: {{ .Values.resources.limits.cpu }}
    memory: {{ .Values.resources.limits.memory }}
```

10.3.3 Déploiement

Puis :

```
git add .
git commit -m "add hpa"
git push
```

👉 et Argo CD fait le reste

10.3.4 Vérification

kubectl get hpa

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get hpa
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
myapp-hpa     Deployment/myapp    cpu: <unknown>/50%  1         5         2         19s
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

Si on a un CPU unknown alors j'ai utilisé la commande ci-dessous :

Kubectl get applications -n argocd

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get applications -n argocd
NAME      SYNC STATUS   HEALTH STATUS
myapp     OutOfSync     Healthy
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

Et je vérifie dans quel état est l'application myapp

Je constate qu'elle est dans l'état : OutOfSync alors qu'elle devrait être dans l'état : Sync

Ce qui explique que les métriques CPU ne sont pas transmises au container de l'application et qu'on a un unknown pour le CPU.

Alors faire : Supprimer puis recréer l'application

kubectl delete application myapp -n argocd

kubectl apply -f argocd/app.yaml

puis vérifier : kubectl get applications -n argocd

ou bien : forcer argocd :

kubectl annotate application myapp -n argocd argocd.argoproj.io/refresh=hard

on vérifie à nouveau : **kubectl get applications -n argocd**

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl annotate application myapp -n argocd argocd.argoproj.io/refresh=hard
application.argoproj.io/myapp annotated
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get applications -n argocd
NAME      SYNC STATUS   HEALTH STATUS
myapp     Synced        Healthy
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

Et maintenant, on constate que le status est : Synced

A nouveau faire : **kubectl get hpa**

```

henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get hpa
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
myapp-hpa     Deployment/myapp    cpu: 0%/50%     1         5         2         117m
henri@Henri:~/labs/eks-terraform-gitops-platform$
henri@Henri:~/labs/eks-terraform-gitops-platform$

```

Et maintenant, ça marche !!. On constate que le CPU est 0%/50% . donc la mtrique est affichée.

Test final : kubectl describe pod <un-pod-myapp>

Avec <un-pod-myapp> - le nom d'un des pods

Par exemple , ici on a : <un-pod-myapp> = myapp-7bb784c447-c5f4r

Commande : **kubectl describe pod myapp-7bb784c447-c5f4r**

Donc : dans le résultat de cette commande , qui est une commande qui liste les propriétés du container (pod) on devrait voir les métriques affichées : on devrait avoir les champs : request et cpu

```

Controlled By: ReplicaSet/myapp-7bb784c447
Containers:
  myapp:
    Container ID:   containerd://413c9400b8094b041aa670a6e408589839ea94be4be2154c5b407c56c2a50635
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:6e23479198b998e5e25921dfff8455837c7636a67111a04a635cf1bb363d199dc
    Port:          80/TCP
    Host Port:     0/TCP
    State:         Running
      Started:     Mon, 04 May 2026 22:30:28 -0400
    Ready:         True
    Restart Count:  0
    Limits:
      cpu:         200m
      memory:      256Mi
    Requests:
      cpu:         100m
      memory:      128Mi
    Environment:   <none>

```

10.3.5 Test réel

Générer du trafic

```

kubectl run load-generator \
  --image=busybox \
  --restart=Never \
  -- /bin/sh -c "while true; do wget -q -O- http://myapp; done"

```

Lancer plusieurs generateurs si nécessaires :

```

kubectl run load1 --image=busybox --restart=Never -- /bin/sh -c "while true; do wget -q -O- http://myapp; done"

```

```

kubectl run load2 --image=busybox --restart=Never -- /bin/sh -c "while true; do wget -q -O- http://myapp; done"

```

On constate 5 pods qui sont en fonction

```
henri@Henri:~/labs/eks-terraform-gitops-platform$  
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods  
NAME                READY   STATUS    RESTARTS   AGE  
load-generator      1/1     Running   0           55m  
load1               1/1     Running   0           2m39s  
load2               1/1     Running   0           2m38s  
myapp-84b7dd7bdc-7t2qc 1/1     Running   0           127m  
myapp-84b7dd7bdc-mg7hg 1/1     Running   0           127m  
henri@Henri:~/labs/eks-terraform-gitops-platform$
```

10.3.6 Observer le scaling

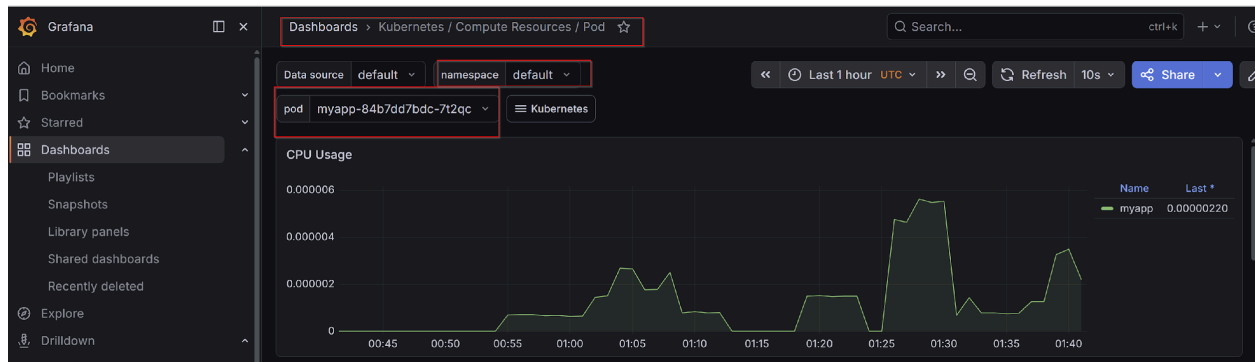
```
kubectl get hpa -w  
kubectl get pods -w
```

👉 tu dois voir :
replicas ↑
pods ↑

On constate ci-dessous que quand le CPU augmente , le nombre de replicas s'adapte, il croit ou décroît en fonction de la charge CPU. Doc le nombre de pods croit ou décroît (principe de HPA) pour s'adaper a la charge .

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get hpa -w  
NAME                REFERENCE          TARGETS          MINPODS   MAXPODS   REPLICAS   AGE  
myapp-hpa           Deployment/myapp    cpu: 0%/50%     1          5          2          127m  
myapp-hpa           Deployment/myapp    cpu: 1%/50%     1          5          2          128m  
myapp-hpa           Deployment/myapp    cpu: 0%/50%     1          5          2          128m  
myapp-hpa           Deployment/myapp    cpu: 1%/50%     1          5          2          130m  
myapp-hpa           Deployment/myapp    cpu: 0%/50%     1          5          2          130m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          130m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          130m  
myapp-hpa           Deployment/myapp    cpu: 1%/5%      1          5          2          132m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          132m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          135m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          135m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          136m  
myapp-hpa           Deployment/myapp    cpu: 1%/5%      1          5          2          138m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          139m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          2          140m  
myapp-hpa           Deployment/myapp    cpu: 1%/5%      1          5          1          141m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          1          141m  
myapp-hpa           Deployment/myapp    cpu: 14%/5%     1          5          1          141m  
myapp-hpa           Deployment/myapp    cpu: 54%/5%     1          5          3          142m  
myapp-hpa           Deployment/myapp    cpu: 13%/5%     1          5          5          142m  
myapp-hpa           Deployment/myapp    cpu: 1%/5%      1          5          5          142m  
myapp-hpa           Deployment/myapp    cpu: 0%/5%      1          5          5          142m
```

pod 1 :



Pod 2 :

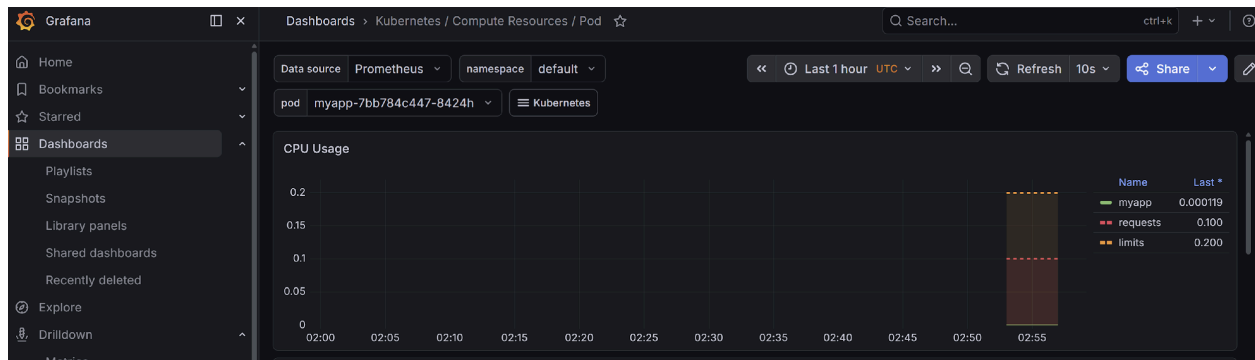
Autre méthode : remplacer l'image Nginx par image: vish/stress

Puis :

- (1) : `kubectl exec -it <nom-du-pod> -- stress --cpu 2 --timeout 60`
- (2) Avec notre exemple :
- (3) `kubectl exec -it -- stress --cpu 2 --timeout 60`

avant d'exécuter (3) : on a :

pod : myapp-7bb784c447-cj8sl



Ensuite après execution de (3) on a :

pod : myapp-7bb784c447-cj8sl

```
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	146m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	147m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	147m
nyapp-hpa	Deployment/myapp	cpu: 1%/5%	1	5	1	147m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	1	148m
nyapp-hpa	Deployment/myapp	cpu: 79%/5%	1	5	1	149m
nyapp-hpa	Deployment/myapp	cpu: 79%/5%	1	5	4	149m
nyapp-hpa	Deployment/myapp	cpu: 17%/5%	1	5	5	149m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	150m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	154m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	1	155m
nyapp-hpa	Deployment/myapp	cpu: 37%/5%	1	5	1	155m
nyapp-hpa	Deployment/myapp	cpu: 45%/5%	1	5	4	155m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	155m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	160m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	160m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	1	161m
nyapp-hpa	Deployment/myapp	cpu: 59%/5%	1	5	1	162m
nyapp-hpa	Deployment/myapp	cpu: 52%/5%	1	5	4	162m
nyapp-hpa	Deployment/myapp	cpu: 1%/5%	1	5	5	162m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	163m
nyapp-hpa	Deployment/myapp	cpu: 0%/5%	1	5	5	167m

>

11. Tests & validations

11.1 Test de résilience

Supprimer un pod

kubectl delete pod <pod>

par exemple : kubectl delete pod myapp-7bb784c447-2bgfh


```

henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
load-generator      1/1     Running   0           161m
load1               1/1     Running   0           108m
load15             1/1     Running   0           21m
load2              1/1     Running   0           108m
load4              1/1     Running   0           52m
load5              1/1     Running   0           49m
myapp-7bb784c447-2bgfh 1/1     Running   0           2m55s
myapp-7bb784c447-g69v2 1/1     Running   0           3m10s
myapp-7bb784c447-jjvbw 1/1     Running   0           3m10s
myapp-7bb784c447-kbfxb 1/1     Running   0           30m
myapp-7bb784c447-n2zxx 1/1     Running   0           3m10s
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl delete pod myapp-7bb784c447-2bgfh
pod "myapp-7bb784c447-2bgfh" deleted from default namespace
henri@Henri:~/labs/eks-terraform-gitops-platform$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
load-generator      1/1     Running   0           162m
load1               1/1     Running   0           109m
load15             1/1     Running   0           21m
load2              1/1     Running   0           109m
load4              1/1     Running   0           52m
load5              1/1     Running   0           49m
myapp-7bb784c447-g69v2 1/1     Running   0           3m57s
myapp-7bb784c447-hvmvs 1/1     Running   0           4s
myapp-7bb784c447-jjvbw 1/1     Running   0           3m57s
myapp-7bb784c447-kbfxb 1/1     Running   0           30m
myapp-7bb784c447-n2zxx 1/1     Running   0           3m57s
henri@Henri:~/labs/eks-terraform-gitops-platform$

```

👉 Résultat attendu :
 nouveau pod recréé automatiquement
 Toujours 5 pods présents